

Agentic Artificial Intelligence

Chapter 04: Multi-Agent Systems

When One Agent Is Not Enough



Prof. Dr. Thomas Clemen

BT7, R. 11.83 | thomas.clemen@haw-hamburg.de

Review + Questions

Semester Plan

Chapter 1. Agents and their Fundamentals

Chapter 2. Perception, Memory, Planning & Learning

Chapter 3. LLMs, RAG & Tool Use

Chapter 4. Multi-Agent Systems

Chapter 5. Agentic AI Engineering

Chapter 6. Safety & Guardrails

The Overwhelmed Single Agent

What happens when one agent gets too many tools?

"Plan a one-week business trip Hamburg → Tokyo: *flights, hotels, meetings with 3 partners, visa check, expense report, local restaurants for business dinner.*"



Unreliable

Too many decisions per ReAct step –
wrong tool selection increases



Slow

Sequential processing – no
parallelism across independent
subtasks



Opaque

When something fails, unclear which
step caused the error

Key question: What if instead of one generalist, we had multiple specialists?

Three Problems → Three Solutions

Problem		Solution		Lecture Part
Who gives orders?	→	Orchestration (Orchestrator/Worker)	→	Part 2: Architectures
How do agents communicate?	→	Communication (Messages, Shared State)	→	Part 3: Communication
How is work distributed?	→	Task Decomposition + Specialization	→	Part 4: Decomposition

Central thesis: A single agent with many tools does not scale indefinitely. Beyond a certain complexity, it is better to distribute the task across specialized agents – but this requires coordination, communication, and clear responsibilities. Multi-agent systems are not mere multiplication, but a distinct architectural decision.

PART 1

From Single-Agent to Multi-Agent

Why one agent is not always enough



What is a Multi-Agent System?



Multiple Autonomous Agents

Each agent has its own perception, memory, planning, and action capabilities



Shared or Distributed Environment

Agents operate in a common environment they can observe and affect



Interaction

Agents cooperate, compete, or do both – they are not independent



No Single Omniscient Agent

No individual agent has complete knowledge or full control over the system

Why Multi-Agent?

Four reasons to distribute tasks across multiple agents



Specialization

Each agent is an expert for one task

Example: Flight agent, Hotel agent, Calendar agent



Parallelism

Independent subtasks run simultaneously

Example: Flight + hotel search in parallel



Robustness

One agent failing $\neq$ system failure

Example: Hotel agent fails $\rightarrow$ travel agent continues



Scalability

New capability = new agent, no rebuild

Example: Add a restaurant agent later

The Coordination Spectrum



Recall Ch. 01: ABM vs. Agentic AI

In ABM (MARS), millions of simple agents communicate implicitly through the environment. In Agentic AI, a few complex agents communicate explicitly via messages. Both are valid forms of multi-agent coordination – but they serve fundamentally different purposes.

PART 2

Architectures for Multi-Agent Systems

Orchestrator/Worker, Peer-to-Peer, Market-Based



Orchestrator / Worker Pattern

The dominant pattern for LLM-based multi-agent systems



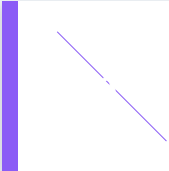
Orchestrator

Receives task, decomposes it, delegates subtasks, collects results, synthesizes final answer

Workers

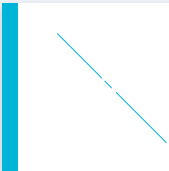
Specialized agents with own system prompt, own toolset, possibly own LLM. Execute one type of task.

Orchestrator in Action: Travel Agent



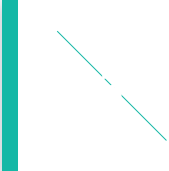
User Request

"Plan business trip Hamburg → Tokyo"



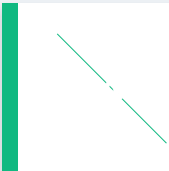
Orchestrator Decomposes

Flights | Hotels | Meetings | Visa



Workers Execute

Each worker uses own tools in parallel



Orchestrator Synthesizes

Consistency check → Final itinerary

The orchestrator is itself a ReAct agent (Ch. 03) – but its "tools" are other agents.

The Orchestrator's Cognitive Architecture

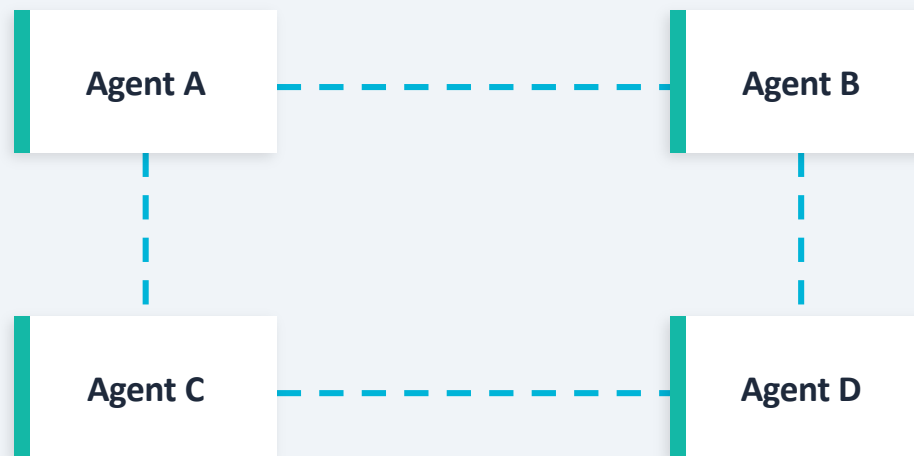
Applying the 4-layer model from Ch. 01 to the orchestrator role

Layer	Orchestrator Realization
Perception	User request + worker results + error signals
Memory	Overall plan + intermediate results + constraints + worker capabilities
Planning	Task decomposition + dependency detection + replanning on failure
Action	Delegation to workers + result synthesis + user response

The orchestrator is a meta-agent: it uses the same cognitive architecture as any LLM-based agent, but its actions are delegations rather than API calls. Its "tools" are worker agents.

Peer-to-Peer / Decentralized Systems

No central coordinator – agents communicate directly



No single point of failure



Harder to debug and trace decisions



Best for equal-status agents (debate, peer review, negotiation)

Market-Based / Auction Systems

Tasks are "advertised" – agents "bid" based on their competence



Contract Net Protocol (Smith, 1980)

Dynamic task allocation based on agent capabilities. Common in robotics and resource allocation. Rarely used in current LLM-based systems – but conceptually important for understanding decentralized coordination.

Architecture Comparison

Property	Orchestrator/Worker	Peer-to-Peer	Market-Based
Coordination	Central	Decentralized	Mechanism-based
Scalability	Good (add workers)	Medium (comm. grows)	Good (add bidders)
Fault Tolerance	Single point of failure	High	Medium
Transparency	High (orchestrator logs)	Low	Medium
Typical Use	LLM-based agent systems	Debate, review, negotiation	Resource allocation, robotics
Complexity	Medium	High	High

Recall Ch. 02: The orchestrator uses deliberative planning; simple workers can be more reactive. The planning spectrum maps directly onto MAS roles.

PART 3

Communication Between Agents

How agents exchange information – and why it's hard



Three Communication Forms



Message Passing

Agent sends structured message to another agent directly

Email between colleagues



Shared State / Blackboard

Agents read/write to a common workspace visible to all

Whiteboard in the office



Tool-as-Agent

Orchestrator calls worker agents like function calls

Boss calls employee on the phone

Anatomy of an Agent Message

What must an inter-agent message contain?

```
{
  "from": "orchestrator",
  "to": "flight_agent",
  "task": "Search cheapest flight HAM→NRT,
          Feb 28 - Mar 1",
  "constraints": {
    "max_price": 1200,
    "class": "economy"
  },
  "deadline": "respond_within_steps: 3",
  "context": "Business trip, meetings
             start Mar 1 afternoon"
}
```

Sender / Receiver

Who sends, who receives

Task

Clear description of what to do

Constraints

Boundaries for the subtask

Deadline

Response time expectations

Context

Relevant background information

Shared State vs. Message Passing

Aspect	Message Passing	Shared State
Coupling	Loose	Tight
Consistency	Simpler (explicit)	Race conditions possible
Parallelism	Natural (async)	Needs synchronization
Debugging	Message log	State snapshots
Frameworks	AutoGen, CrewAI	LangGraph (State Graph)

→ Ch. 05 will implement both patterns using real frameworks.

The LLM Communication Challenge

LLM agents communicate in natural language – blessing and curse

Advantage

Flexible, no rigid protocol needed. Agents can negotiate, clarify, and adapt their communication dynamically.

Risk

Misunderstandings, ambiguity, "lost in translation." One agent's output may not parse correctly as another's input.

Solution: Structured Output (Recall Ch. 03)

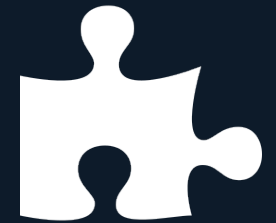
Without reliable JSON, no stable inter-agent communication is possible. Structured Output is not just a convenience – it's the foundation of multi-agent coordination.

→ Ch. 06: Agent-to-agent injection – a new attack vector unique to multi-agent systems.

PART 4

Task Decomposition & Specialization

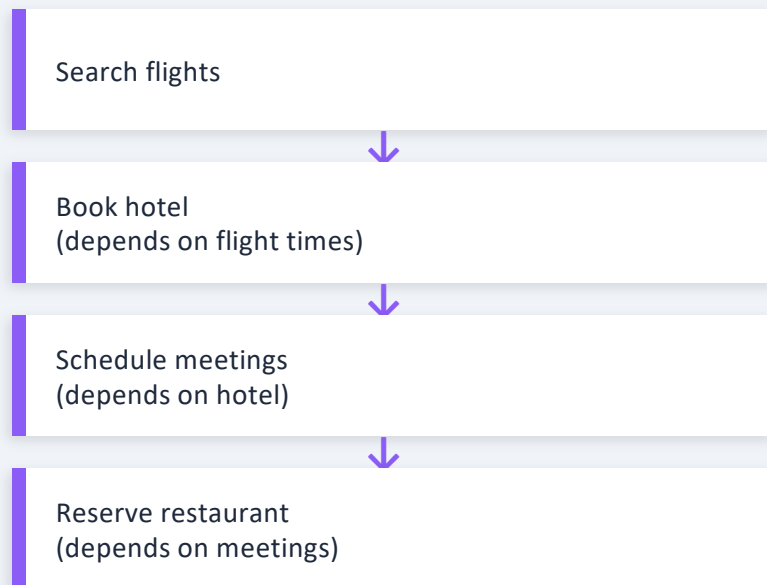
How to break down complex tasks and define agent roles



Sequential vs. Parallel Decomposition

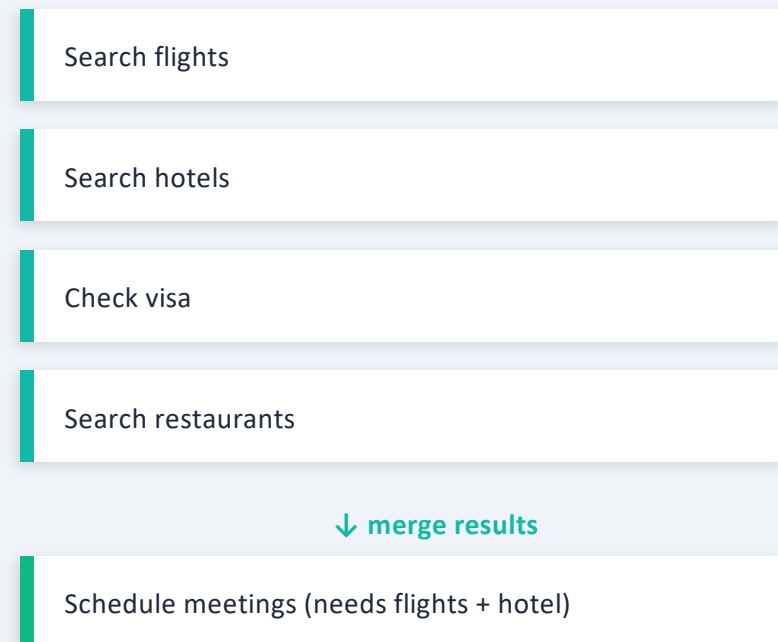
Sequential

Each step depends on the previous one



Parallel

Independent tasks run simultaneously



Defining a Worker Agent

Each worker is defined by three dimensions



System Prompt

Role, responsibilities, and constraints

```
"You are a flight booking specialist.  
Never book without user confirmation."
```



Toolset

Which tools the agent is allowed to use

```
search_flights(), check_availability(),  
book_flight()
```



Scope

Which tasks the agent should and may handle

```
Only flights. No hotels, no meetings,  
no restaurants.
```

Agent specialization is a design decision – narrower scope = more reliable but less flexible.

When Multi-Agent? – Decision Heuristic

Criterion	Single Agent Better	Multi-Agent Better
Task Complexity	Few tools (< 5)	Many specialized subtasks (8–10+ tools)
Parallelism	Everything sequential	Independent subtasks exist
Fault Tolerance	A single error is tolerable	Partial failures must be isolated
Transparency	Simple workflow	Complex chains need traceability
Dev Team Size	One developer	Multiple teams want independence

Multi-Agent is not always better. More agents = more coordination overhead. The decision is a trade-off between capability and complexity.

PART 5

Integration

The big picture – across four chapters



The Travel Agent – Across Four Chapters

Layer	Ch. 01 Abstract	Ch. 02 Mechanisms	Ch. 03 Realization	Ch. 04 Multi-Agent
Perception	PEAS: User prefs, search results	Raw data → Preprocessing → State	Prompt + RAG context + API responses	Orchestrator: User request + worker results
Memory	"Has access to calendar"	Working, Semantic, Episodic	Context Window + Vector DB + Chat Log	Shared State + agent-specific memory
Planning	"Compares options"	Deliberative, hierarchical decomp.	ReAct loop: Think → Act → Observe	Task decomposition + delegation + replanning
Action	PEAS: Initiate booking	Capability, granularity, irreversibility	Function calls: search(), book()	Orchestrator delegates, workers execute

From abstract → mechanisms → concrete technology → distributed system. The same agent, increasingly realized.

Counterexample: MARS LaserTag as MAS

Both use multiple agents – but for fundamentally different reasons

Dimension	LLM-Based MAS (Travel Agent)	ABM-MAS (MARS LaserTag)
# of Agents	4–10 specialized	Hundreds to thousands, homogeneous
Communication	Explicit (messages, shared state)	Implicit (through the environment)
Coordination	Orchestrator plans centrally	Emergent (no coordinator)
Agent Complexity	High (own LLM, own tools)	Low (few rules)
Goal	Solve a task	Understand behavior
Emergence	Undesired (= loss of control)	Desired (= insight)

Recall Ch. 01: ABM asks "What emerges?" – Agentic AI asks "What should emerge, and how do we prevent everything else?" This distinction repeats at the MAS level.

Excursus: Multi-Agent Reinforcement Learning

Optional – conceptual overview without RL prerequisites



Cooperative

All agents share one reward signal

Robot team carrying object together



Competitive

One agent's reward = another's penalty

Chess, Go, StarCraft



Mixed

Cooperate within team, compete between teams

Football (team cooperates, teams compete)

Connection to LLM-based MAS:

Current LLM-MAS do not use RL for coordination (too expensive, too unstable). But the coordination problem is the same: how do agents align when the "environment" (= other agents) keeps changing? Research frontier: Can LLM agents learn to cooperate better through interaction?

Key Takeaways

1

Multi-agent systems solve the scaling problem.

One agent with 20 tools $\neq$ four agents with 5 tools each.

2

Orchestrator/Worker is the dominant pattern.

The orchestrator plans and delegates, workers execute.

3

Communication is the critical infrastructure.

Message Passing vs. Shared State – the choice determines coupling and debuggability.

4

Task decomposition is an architecture decision.

Sequential vs. parallel, dependent vs. independent.

5

Multi-agent is not always better.

More agents = more coordination overhead. The decision is a trade-off.

Preview: Chapter 05

Agentic AI Engineering

We now know what a multi-agent system is and how it's organized.
But how do you actually build one?

Which frameworks exist?

LangGraph, AutoGen, CrewAI – which one for which use case?

How do you manage state?

State graphs, checkpoints, memory across agents?

How do you test non-deterministic systems?

When the same input can produce different outputs?

References & Further Reading

Wooldridge, M. (2009): "An Introduction to MultiAgent Systems" (2nd Edition)

Park, J. S. et al. (2023): "Generative Agents: Interactive Simulacra of Human Behavior"

Wu, Q. et al. (2023): "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation"

Hong, S. et al. (2023): "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework"

Li, G. et al. (2023): "CAMEL: Communicative Agents for Mind Exploration of Large Scale Language Model Society"

Dorri, A. et al. (2018): "Multi-Agent Systems: A Survey" – IEEE Access

Smith, R. G. (1980): "The Contract Net Protocol" – IEEE Transactions